

Mini Project-1
SQL project: Retail Store Sales Analysis

By
Yarmik Kansagara

-- Step 1: Create Database

```
CREATE DATABASE smartmart_db;
```

```
USE smartmart_db;
```

-- Step 2: Create Tables

--1. Customers Table

```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(50),  
    city VARCHAR(50),  
    age INT  
);
```

-- 2.Products Table

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100),  
    category VARCHAR(50),  
    price DECIMAL(10,2)  
);
```

-- 3.Sales Table

```
CREATE TABLE sales (  
    order_id INT PRIMARY KEY,  
    customer_id INT,  
    product_id INT,  
    quantity INT,
```

```
order_date DATE);
```

-- Step 3: Insert Sample Data

-- Customers

```
INSERT INTO customers VALUES
```

```
(1,'Rahul Sharma','Delhi',28),  
(2,'Priya Verma','Mumbai',32),  
(3,'Amit Singh','Bangalore',25),  
(4,'Neha Kapoor','Delhi',29),  
(5,'Rohit Mehta','Pune',35),  
(6,'Anjali Gupta','Mumbai',27),  
(7,'Karan Patel','Ahmedabad',31),  
(8,'Sneha Reddy','Hyderabad',26);
```

-- Products

```
INSERT INTO products VALUES
```

```
(101,'Laptop','Electronics',80000),  
(102,'Smartphone','Electronics',40000),  
(103,'Headphones','Electronics',2000),  
(104,'T-Shirt','Clothing',800),  
(105,'Jeans','Clothing',2000),  
(106,'Microwave','Home Appliances',7000),  
(107,'Refrigerator','Home Appliances',30000),  
(108,'Air Conditioner','Home Appliances',45000);
```

-- Sales

INSERT INTO sales VALUES

(1001,1,101,1,'2024-01-10'),
(1002,2,102,2,'2024-01-11'),
(1003,3,104,3,'2024-01-12'),
(1004,4,105,2,'2024-01-12'),
(1005,5,103,4,'2024-01-13'),
(1006,6,106,1,'2024-01-14'),
(1007,7,108,1,'2024-01-14'),
(1008,8,102,1,'2024-01-15'),
(1009,1,103,2,'2024-01-16'),
(1010,2,104,5,'2024-01-17'),
(1011,3,105,1,'2024-01-18'),
(1012,4,101,1,'2024-01-19');

Practice Questions

Level 1 – Basic SELECT

1) Display all customers.

➔ select * from customers;

```
79      -- Display all customers.
80 •    select * from customers;
81 ✖    Show only customer_name and city.
82
```

Result Grid | Filter Rows: | Edit:

	customer_id	customer_name	city	age
▶	1	Rahul Sharma	Delhi	28
	2	Priya Verma	Mumbai	32
	3	Amit Singh	Bangalore	25
	4	Neha Kapoor	Delhi	29
	5	Rohit Mehta	Pune	35
	6	Anjali Gupta	Mumbai	27
	7	Karan Patel	Ahmedabad	31
	8	Sneha Reddy	Hyderabad	26
*	NULL	NULL	NULL	NULL

2) Show only customer_name and city.

➔ select customer_name,city from customers;

```
81
82      -- Show only customer_name and city.
83 •    select customer_name,city from customers;
84
```

Result Grid | Filter Rows: | Export: | Wrap C

	customer_name	city
▶	Rahul Sharma	Delhi
	Priya Verma	Mumbai
	Amit Singh	Bangalore
	Neha Kapoor	Delhi
	Rohit Mehta	Pune
	Anjali Gupta	Mumbai
	Karan Patel	Ahmedabad
	Sneha Reddy	Hyderabad

3) Display all products in the Electronics category.

➔ select * from products where category='Electronics';

```
84
85  -- Display all products in the Electronics category.
86 • select * from products where category='Electronics';
87
```

Result Grid	Filter Rows:	Edit:	Export/Import:
product_id	product_name	category	price
101	Laptop	Electronics	80000.00
102	Smartphone	Electronics	40000.00
103	Headphones	Electronics	2000.00
NULL	NULL	NULL	NULL

4) Show products with price greater than 10,000.

➔ select * from products where price > 10000;

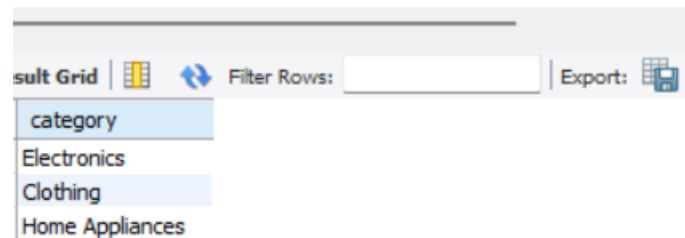
```
88
89  -- Show products with price greater than 10,000.
90 • select * from products where price > 10000;
91
```

Result Grid	Filter Rows:	Edit:	Export/Import:
product_id	product_name	category	price
101	Laptop	Electronics	80000.00
102	Smartphone	Electronics	40000.00
107	Refrigerator	Home Appliances	30000.00
108	Air Conditioner	Home Appliances	45000.00
NULL	NULL	NULL	NULL

5) Display unique categories.

➔ select distinct category from products;

```
12 -- Display unique categories.
13 • select distinct category from products;
```



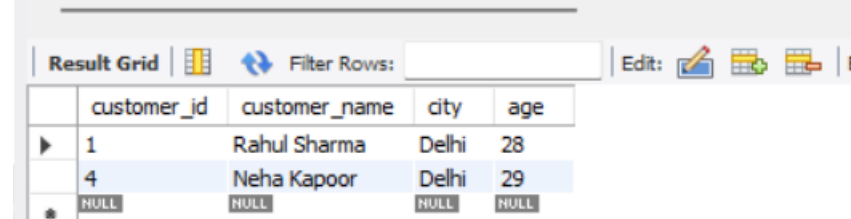
category
Electronics
Clothing
Home Appliances

Level 2 – WHERE Conditions

6. Show customers who live in Delhi.

➔ select * from customers where city = 'Delhi';

```
98 -- 6. Show customers who live in Delhi.
99 • select * from customers where city = 'Delhi';
100 ✖ 7. Show customers with age greater than 30.
```



	customer_id	customer_name	city	age
▶	1	Rahul Sharma	Delhi	28
	4	Neha Kapoor	Delhi	29
*	NULL	NULL	NULL	NULL

7. Show customers with age greater than 30.

➔ select * from customers where age > 30;

```
101      -- 7.  Show customers with age greater than 30.
102 •    select * from customers where age > 30;
103
```

	customer_id	customer_name	city	age
▶	2	Priya Verma	Mumbai	32
	5	Rohit Mehta	Pune	35
	7	Karan Patel	Ahmedabad	31
*	NULL	NULL	NULL	NULL

8. Display products with price between 2000 and 50000.

➔ select * from products where price between 2000 and 50000;

```
104      -- 8.  Display products with price between 2000 and 50000.
105 •    select * from products where price between 2000 and 50000;
106
```

	product_id	product_name	category	price
▶	102	Smartphone	Electronics	40000.00
	103	Headphones	Electronics	2000.00
	105	Jeans	Clothing	2000.00
	106	Microwave	Home Appliances	7000.00
	107	Refrigerator	Home Appliances	30000.00
	108	Air Conditioner	Home Appliances	45000.00
*	NULL	NULL	NULL	NULL

9. Find orders placed after 2024-01-12.

➔ select * from sales where order_date > '2024-01-12';

```
107 -- 9. Find orders placed after 2024-01-12.
108 • select * from sales where order_date > '2024-01-12';
109
```

Result Grid

Filter Rows:

Edit:

Export/Import:

	order_id	customer_id	product_id	quantity	order_date
▶	1005	5	103	4	2024-01-13
	1006	6	106	1	2024-01-14
	1007	7	108	1	2024-01-14
	1008	8	102	1	2024-01-15
	1009	1	103	2	2024-01-16
	1010	2	104	5	2024-01-17
	1011	3	105	1	2024-01-18
	1012	4	101	1	2024-01-19
•	NULL	NULL	NULL	NULL	NULL

sales 10 x

10. Display sales where quantity > 2.

➔ select * from sales where quantity > 2;

```
109
110     -- 10. Display sales where quantity > 2.
111 • select * from sales where quantity > 2;
112
```

Result Grid

  Filter Rows:

Edit:    Export/Impx

	order_id	customer_id	product_id	quantity	order_date
▶	1003	3	104	3	2024-01-12
	1005	5	103	4	2024-01-13
	1010	2	104	5	2024-01-17
*	NULL	NULL	NULL	NULL	NULL

Level 3 – Sorting & Limiting

11. Display products sorted by price (highest first).

➔ `SELECT * FROM products ORDER BY price DESC;`

```
114 -- 11. Display products sorted by price (highest first).
115 • SELECT * FROM products ORDER BY price DESC;
116 -- 12. Show top 3 most expensive products.
```

	product_id	product_name	category	price
▶	101	Laptop	Electronics	80000.00
	108	Air Conditioner	Home Appliances	45000.00
	102	Smartphone	Electronics	40000.00
	107	Refrigerator	Home Appliances	30000.00
	106	Microwave	Home Appliances	7000.00
	103	Headphones	Electronics	2000.00
	105	Jeans	Clothing	2000.00
	104	T-Shirt	Clothing	800.00
*	NULL	NULL	NULL	NULL

12. Show top 3 most expensive products.

➔ `select * from products order by price desc limit 3;`

```
116 -- 12. Show top 3 most expensive products.
117 • select * from products order by price desc limit 3;
118 -- 13. Display customers sorted by age ascending.
```

	product_id	product_name	category	price
▶	101	Laptop	Electronics	80000.00
	108	Air Conditioner	Home Appliances	45000.00
	102	Smartphone	Electronics	40000.00
*	NULL	NULL	NULL	NULL

13. Display customers sorted by age ascending.

➔ select * from customers order by age asc;

```
119 • select * from customers order by age asc;
```

```
120 -- 14. Show latest 5 orders.
```

Result Grid			
Filter Rows:			
Edit:   			
customer_id	customer_name	city	age
3	Amit Singh	Bangalore	25
8	Sneha Reddy	Hyderabad	26
6	Anjali Gupta	Mumbai	27
1	Rahul Sharma	Delhi	28
4	Neha Kapoor	Delhi	29
7	Karan Patel	Ahmedabad	31
2	Priya Verma	Mumbai	32
5	Rohit Mehta	Pune	35
NULL	NULL	NULL	NULL

customers 16 x

14. Show latest 5 orders.

➔ select * from sales order by order_date desc; limit 5;

```
120 -- 14. SHOW LATEST 5 ORDERS.
```

```
121 • select * from sales order by order_date desc limit 5;
```

Result Grid					
Filter Rows:					
Edit:    Export/Import: 					
	order_id	customer_id	product_id	quantity	order_date
▶	1012	4	101	1	2024-01-19
	1011	3	105	1	2024-01-18
	1010	2	104	5	2024-01-17
	1009	1	103	2	2024-01-16
	1008	8	102	1	2024-01-15
✱	NULL	NULL	NULL	NULL	NULL

Level 4 – NULL Handling (Practice)

Update a row to create NULL values.

UPDATE customers

SET city = NULL

WHERE customer_id = 8;

Questions:

15. Find customers where city IS NULL

➔ select * from customers where city is null;

```
128 • select * from customers where city is null;
```

Result Grid

  Filter Rows:

Edit:   

	customer_id	customer_name	city	age
▶	8	Sneha Reddy	NULL	26
•	NULL	NULL	NULL	NULL

16. Find customers where city IS NOT NULL

➔ select * from customers where city is not null;

```
130 -- 16. Find customers where city IS NOT NULL
131 • select * from customers where city is not null;
```

Result Grid

  Filter Rows:

Edit:   

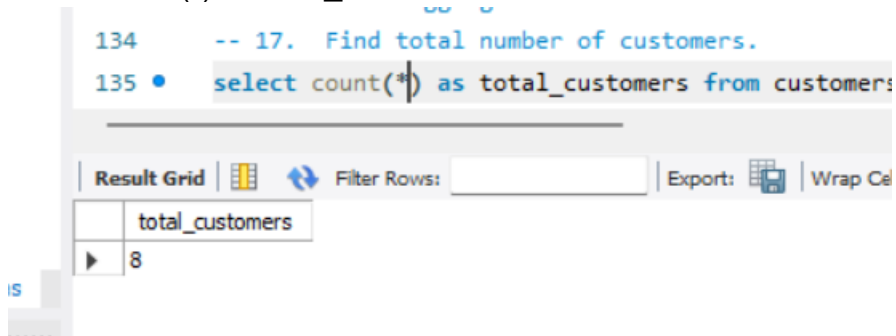
	customer_id	customer_name	city	age
▶	1	Rahul Sharma	Delhi	28
	2	Priya Verma	Mumbai	32
	3	Amit Singh	Bangalore	25
	4	Neha Kapoor	Delhi	29
	5	Rohit Mehta	Pune	35
	6	Anjali Gupta	Mumbai	27
	7	Karan Patel	Ahmedabad	31
●	NULL	NULL	NULL	NULL

Level 5 – Aggregations

17. Find total number of customers.

➔ select count(*) as total_customers from customers;

```
134      -- 17. Find total number of customers.
135 •    select count(*) as total_customers from customers;
```

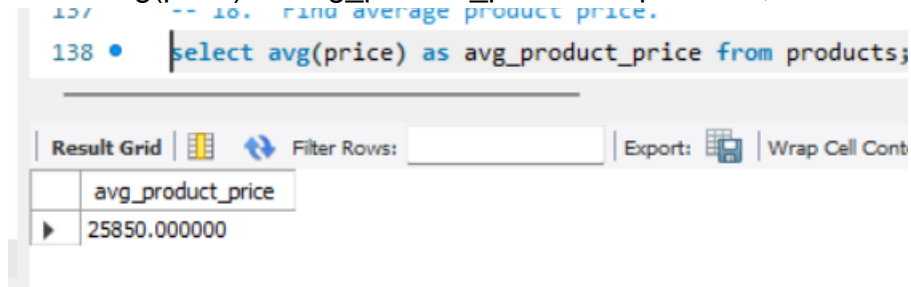


total_customers
8

18. Find average product price.

➔ select avg(price) as avg_product_price from products;

```
137      -- 18. Find average product price.
138 •    select avg(price) as avg_product_price from products;
```

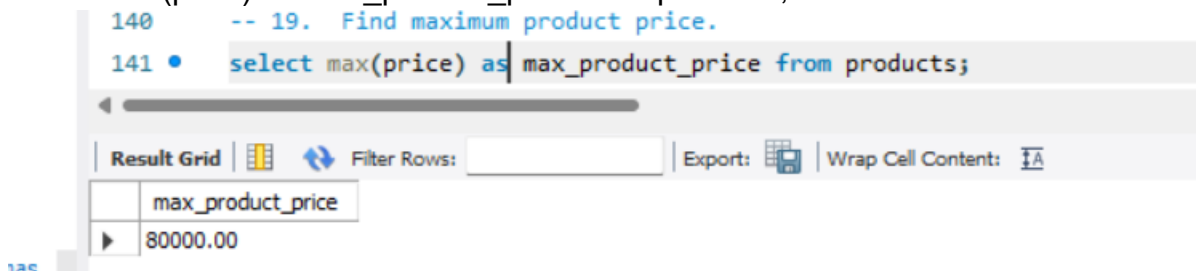


avg_product_price
25850.000000

19. Find maximum product price.

➔ select max(price) as max_product_price from products;

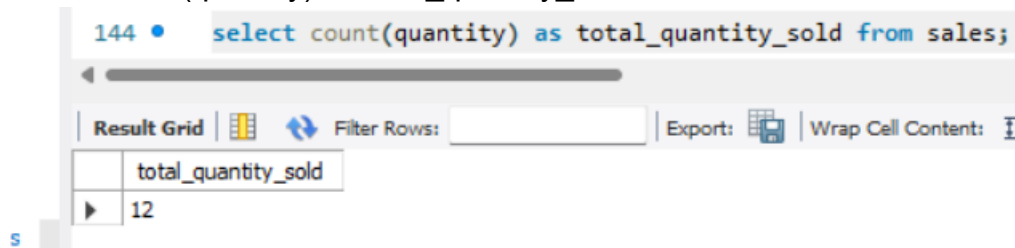
```
140      -- 19. Find maximum product price.
141 •    select max(price) as max_product_price from products;
```



max_product_price
80000.00

20. Find total quantity sold.

➔ select count(quantity) as total_quantity_sold from sales;



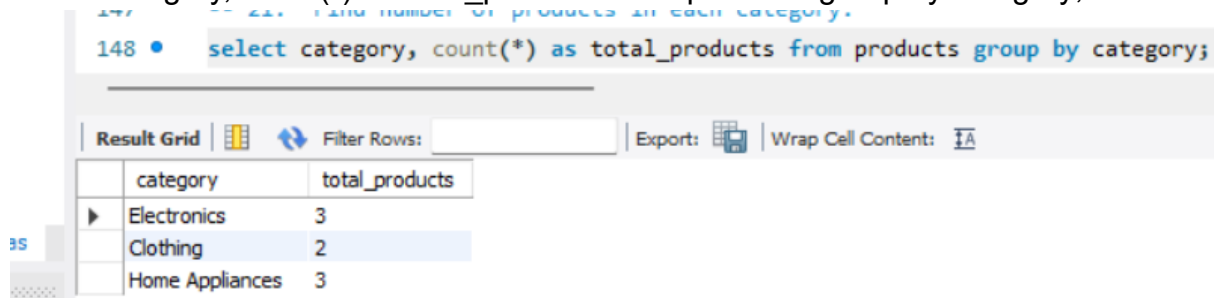
The screenshot shows a SQL query editor with a query window at the top containing the SQL statement: `select count(quantity) as total_quantity_sold from sales;`. Below the query window is a toolbar with options like 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. The 'Result Grid' is active, displaying a single row with the column name 'total_quantity_sold' and the value '12'.

total_quantity_sold
12

Level 6 – GROUP BY

21. Find number of products in each category.

➔ select category, count(*) as total_products from products group by category;



The screenshot shows a SQL query editor with a query window at the top containing the SQL statement: `select category, count(*) as total_products from products group by category;`. Below the query window is a toolbar with options like 'Result Grid', 'Filter Rows', 'Export', and 'Wrap Cell Content'. The 'Result Grid' is active, displaying a table with three rows: Electronics (3), Clothing (2), and Home Appliances (3).

category	total_products
Electronics	3
Clothing	2
Home Appliances	3

22. Find total sales quantity by product_id.

➔ select product_id, count(quantity) as total_sales from sales group by product_id;

```
151 • select product_id, count(quantity) as total_sales from sales group by product_id;
```

product_id	total_sales
101	2
102	2
104	2
105	2
103	2
106	1
108	1

23. Find average price by category.

➔ select category, avg(price) as avg_price from products group by category;

```
53 -- 23. Find average price by category.
```

```
54 • select category, avg(price) as avg_price from products group by category;
```

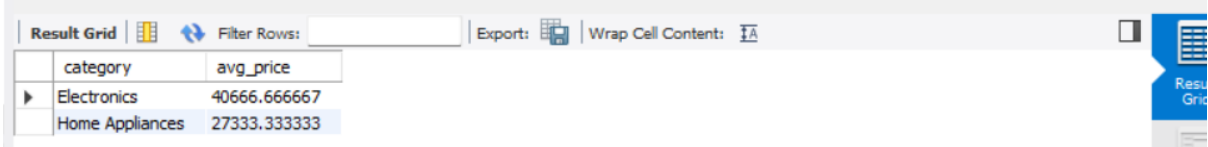
category	avg_price
Electronics	40666.666667
Clothing	1400.000000
Home Appliances	27333.333333

Level 7 – HAVING Clause

24. Find categories where average price > 10,000.

➔ select category, avg(price) as avg_price from products group by category having avg(price) > 10000;

```
157 -- 24. Find categories where average price > 10,000.
158 • select category, avg(price) as avg_price from products group by category having avg(price) > 10000;
```



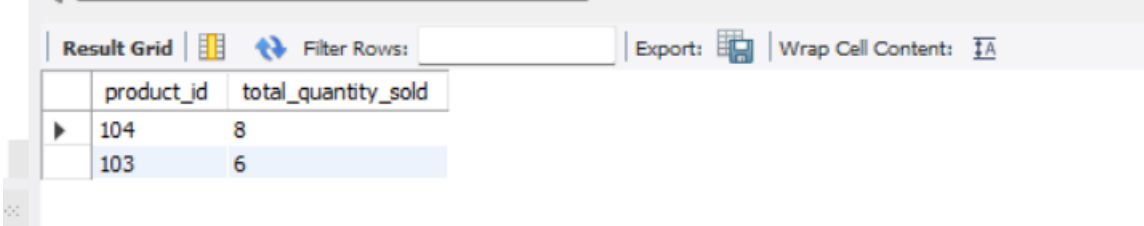
The screenshot shows a database query result grid. The query is: `select category, avg(price) as avg_price from products group by category having avg(price) > 10000;`. The result grid has two columns: `category` and `avg_price`. There are two rows: `Electronics` with `40666.666667` and `Home Appliances` with `27333.333333`.

category	avg_price
Electronics	40666.666667
Home Appliances	27333.333333

25. Find products where total quantity sold > 3.

➔ select product_id, sum(quantity) as total_quantity_sold from sales group by product_id having sum(quantity) > 3;

```
161 • select product_id, sum(quantity) as total_quantity_sold from sales
162   group by product_id having sum(quantity) > 3;
```



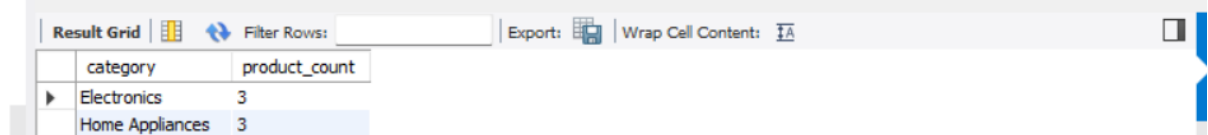
The screenshot shows a database query result grid. The query is: `select product_id, sum(quantity) as total_quantity_sold from sales group by product_id having sum(quantity) > 3;`. The result grid has two columns: `product_id` and `total_quantity_sold`. There are two rows: `104` with `8` and `103` with `6`.

product_id	total_quantity_sold
104	8
103	6

26. Find categories having more than 2 products.

➔ select category, count(*) as product_count from products group by category having count(*) > 2;

```
165 • select category, count(*) as product_count from products group by category having count(*) > 2;
```



The screenshot shows a database query result grid. The query is: `select category, count(*) as product_count from products group by category having count(*) > 2;`. The result grid has two columns: `category` and `product_count`. There are two rows: `Electronics` with `3` and `Home Appliances` with `3`.

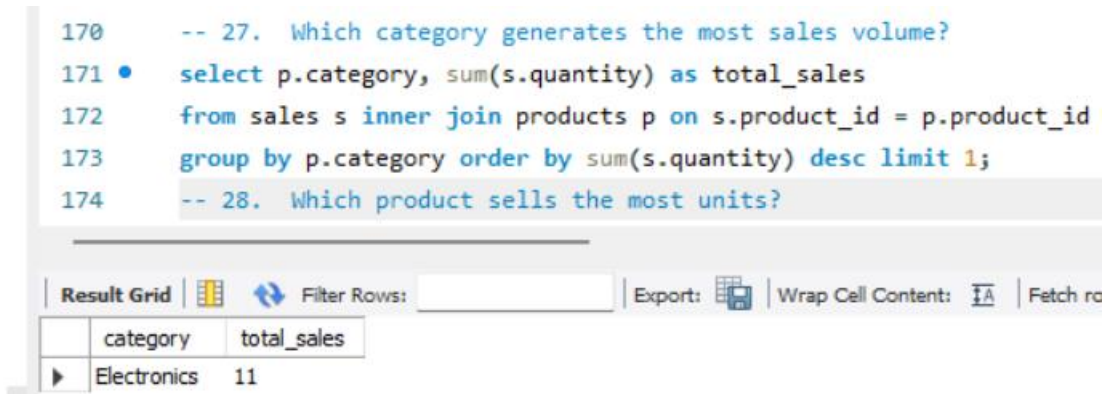
category	product_count
Electronics	3
Home Appliances	3

Business Questions (Real Analyst Thinking)

27. Which category generates the most sales volume?

- ➔ select p.category, sum(s.quantity) as total_sales from sales s inner join products p on s.product_id = p.product_id group by p.category order by sum(s.quantity) desc limit 1;

```
170 -- 27. Which category generates the most sales volume?
171 • select p.category, sum(s.quantity) as total_sales
172 from sales s inner join products p on s.product_id = p.product_id
173 group by p.category order by sum(s.quantity) desc limit 1;
174 -- 28. Which product sells the most units?
```



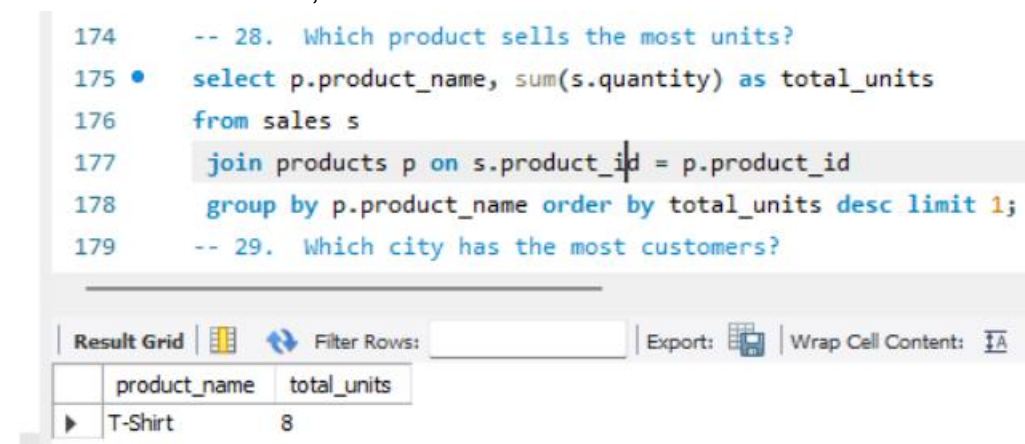
Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows

	category	total_sales
▶	Electronics	11

28. Which product sells the most units?

- ➔ select p.product_name, sum(s.quantity) as total_units from sales s; join products p on s.product_id = p.product_id group by p.product_name order by total_units desc limit 1;

```
174 -- 28. Which product sells the most units?
175 • select p.product_name, sum(s.quantity) as total_units
176 from sales s
177 join products p on s.product_id = p.product_id
178 group by p.product_name order by total_units desc limit 1;
179 -- 29. Which city has the most customers?
```

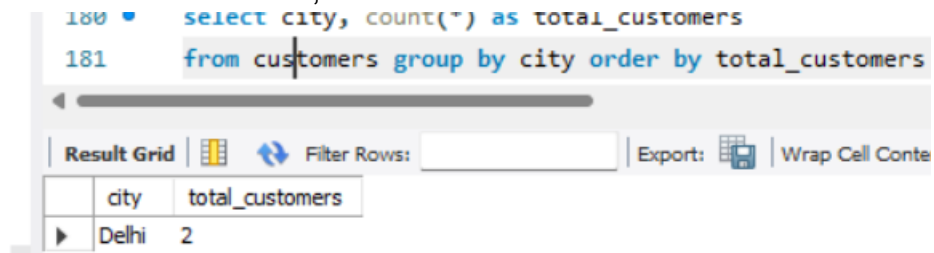


Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows

	product_name	total_units
▶	T-Shirt	8

29. Which city has the most customers?

➔ select city, count(*) as total_customers from customers group by city order by total_customers desc limit 1;



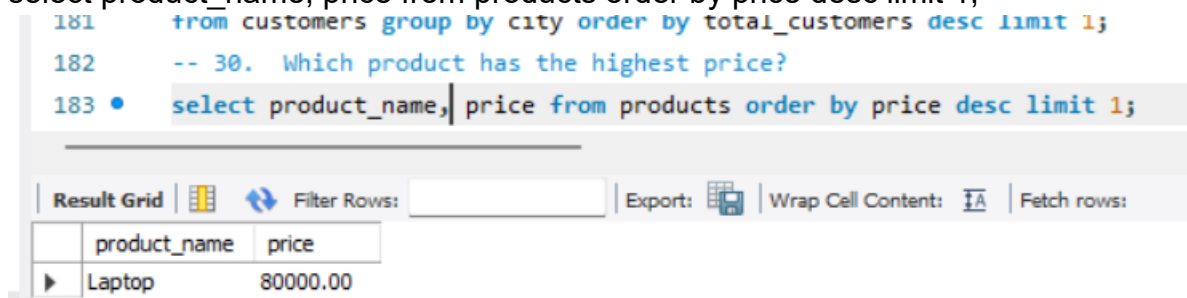
```
180 select city, count(*) as total_customers
181 from customers group by city order by total_customers desc limit 1;
```

Result Grid

city	total_customers
Delhi	2

30. Which product has the highest price?

➔ select product_name, price from products order by price desc limit 1;



```
181 from customers group by city order by total_customers desc limit 1;
182 -- 30. Which product has the highest price?
183 select product_name, price from products order by price desc limit 1;
```

Result Grid

product_name	price
Laptop	80000.00